

# Checkpoint — Análise de Data Drift em Seq2Seq para Predição de Trajetórias no RoboCup SSL

Iniciação Científica — Relatório de progresso (Mês 1 e Mês 2)

April 28, 2026

## Resumo

Este checkpoint documenta as mudanças realizadas no projeto `trajectory-prediction` entre a implementação do Mês 1 e o estado atual da pesquisa. O objetivo do Mês 1 foi estabelecer infraestrutura granular (por jogo, por janela e por trajetória) para substituir a granularidade anual do notebook original. O Mês 2 introduziu detectores formais de drift (ADWIN, Page-Hinkley, KSWIN, Pelt) para trocar a inferência visual por testes estatísticos. Em seguida, uma rodada crítica identificou 19 pontos frágeis no pipeline e implementou as correções prioritárias: ordem temporal das trajetórias, recalibração de detectores, decomposição covariate vs concept drift, null detector via permutação e curva de latência. O notebook unificado tem hoje 55 células e produz 39 figuras em PT/EN além de 12 CSVs de métricas, todos versionados em `Relas/results/{mes1,mes2}/`.

## 1 Linha do tempo

1. **Mês 1 — Infraestrutura granular.** Enriquecimento do `dataset.csv` com contexto cinemático por jogo, expansão para granularidade de janela e amostra de erros por trajetória.
2. **Mês 2 — Detectores formais.** ADWIN, Page-Hinkley, KSWIN, Pelt aplicados sobre os streams definidos no Mês 1.
3. **Iteração crítica.** Análise de 19 pontos fracos; reescrita do notebook unificado e do `plot_covariate_shift.py`; criação de `compute_trajectory_errors.py`.
4. **Estado atual.** Pipeline documentado em `Relas/PIPELINE.md`, dependências fixadas em `requirements.txt`, modelo configurado para target `dv` (delta-velocity), divisão A/B mapeada via `division_map.csv`. Pronto para o Mês 3.

## 2 Mês 1 — Infraestrutura granular

### 2.1 Entregas

- `plot_covariate_shift.py` estendido para gerar dois arquivos novos:
  - `per_game_context.csv` — uma linha por jogo com `match_id`, `log_file`, ano, divisão, `avg_robots_per_stoppage`, número de trajetórias, duração estimada e percentis cinemáticos (`speed_p{50,90,99}`, `accel_p{50,90,99}`, `turn_p{50,90}`).
  - `per_window_drift.csv` — KS e Wasserstein por janela contra o *baseline* de treino (`proc_set_1+2`). Tamanho de janela `N_TRAJS_PER_WINDOW = 30` (calibrado abaixo).
- `compute_trajectory_errors.py` (script novo) — carrega o Seq2Seq treinado e o *Kalman* determinístico, roda numa amostra de 3 jogos por ano (`seed = 42`) e produz `trajectory_errors_sample.par` com ADE/FDE por janela.
- Notebook `drift_analise.ipynb` (Seções 1–5) com descritivas na nova granularidade: ADE por jogo com IC *bootstrap*, ADE vs features cinemáticas, heatmap Spearman, tabelas de resumo e checagens de sanidade.

- Saídas em PT e EN salvas em `Relas/results/mes1/`: 19 figuras PDF (par PT/EN) + três CSVs (`summary_granularity.csv`, `tail_stats.csv`, `window_intra_year_var.csv`).

## 2.2 Decisões de design

- *Granularidade de janela*: a especificação inicial fixava  $N = 500$  trajetórias, o que produzia uma janela por jogo em quase todos os casos — equivalente à granularidade anual do `ks_wd_per_dataset.csv` já existente. Reduzimos para  $N = 30$  após a iteração crítica, obtendo mediana de 6 janelas por jogo (Tabela 1).
- *Ordem temporal*: a versão original de `iter_robot_trajs` concatenava todas as trajetórias do time `blue` antes do `yellow`, sem garantia de ordem cronológica. Foi reescrita para ordenar por `time_c[0]` de cada trajetória.
- *Bilinguismo*: todos os plots são gerados em PT e EN via `T(pt, en, lang)`, salvando o par `<nome>_{pt,en}.pdf` em `Relas/results/mes1/`.

| Ano          | Número de jogos | Trajeto rias totais | Janelas totais |
|--------------|-----------------|---------------------|----------------|
| 2019         | 6               | 1 171               | 42             |
| 2021         | 6               | 2 206               | 77             |
| 2022         | 6               | 807                 | 30             |
| 2023         | 6               | 903                 | 34             |
| 2024         | 6               | 886                 | 31             |
| 2025         | 6               | 1 055               | 39             |
| <b>Total</b> | <b>36</b>       | <b>7 028</b>        | <b>253</b>     |

Table 1: Granularidade obtida com `N_TRAJS_PER_WINDOW = 30`.  
`Relas/results/mes1/summary_granularity.csv`.

Fonte:

## 3 Mês 2 — Detectores formais de drift

### 3.1 Entregas

- **Seção 6 — Concept drift online.** ADWIN (`delta = 0.002`) e Page-Hinkley (`threshold = 5.0 mm`, recalibrado pela ordem de grandeza do ADE) sobre a s rie de `ade_traj` do Seq2Seq 30→15, ordenada por (`year`, `match_id`, `traj_id`). O mesmo procedimento   aplicado ao Kalman determin stico para permitir leitura cruzada Seq2Seq vs Kalman — drift no Kalman sinaliza mudan a no comportamento do alvo, drift apenas no Seq2Seq sinaliza miscalibra  o do modelo.
- **Se  o 7 — Covariate shift online.** KSWIN com `window_size = 30`, `stat_size = 10`, `alpha = 0.005` sobre `ks_speed_window` e `ks_accel_window`. Par metros calibrados para o stream de 253 janelas; com  $N = 500$  original n o havia amostras suficientes para o detector disparar.
- **Se  o 8 — Benchmark offline com Pelt.** `ruptures.Pelt` com custo RBF sobre o ADE m dio por jogo (s rie de 30 pontos). *Elbow plot* variando a penalidade  $pen \in \{1, 2, 3, 5, 7, 10, 15, 20, 30, 50, 75, 100, 150\}$  guia a escolha autom tica de `PEN_CHOSEN`.
- **Se  o 9 — S ntese.** Fun  o `map_idx_to_game` converte  ndices de cada stream em (`year`, `match_id`, `log_file`). Tabela `alarms_consolidated.csv` re ne os alarmes de todos os detectores para discuss o.
- Sa das em `Relas/results/mes2/`.

## 4 Iteração crítica e melhorias incorporadas

A revisão do estado pós-Mês 2 identificou 19 pontos frágeis, agrupados em três categorias: integridade dos dados, calibração estatística e análises faltantes. As melhorias prioritárias foram implementadas:

### 4.1 Integridade dos dados

- *Ordem temporal das trajetórias.* `iter_robot_trajs` agora intercala `blue` e `yellow` ordenando por `time_c[0]` (parâmetro `chrono = True`).
- *Granularidade de janela.*  $N = 500 \rightarrow N = 30$ .
- *Coluna `n_robots`.* Renomeada para `avg_robots_per_stoppage`, refletindo que é uma *proxy* (razão trajetórias / `stop_id`s únicos), não o número real de robôs simultâneos em campo.
- *Modo de predição.* `compute_trajectory_errors.py` fixa `target = "dv"` (delta-velocity) para alinhar com os pesos treinados; o *Kalman* de referência passa a ser  $\hat{\Delta}v = 0$  (velocidade constante implica incremento nulo).

### 4.2 Calibração estatística

- *Page-Hinkley.* `threshold` ajustado de 50 para 5 mm. Para ADE em milímetros com mediana  $\sim 10\text{--}50$  mm, `threshold = 50` era praticamente impossível de disparar. O novo valor torna o detector visível ao ruído real do domínio.
- *KSWIN.* `window_size` ajustado para 30 e `stat_size` para 10, compatíveis com o tamanho efetivo do stream de janelas.
- *Pelt.* Penalidade escolhida via *elbow plot* ao invés de chute fixo.

### 4.3 Análises adicionadas

- **Seção 10.1 — Null detector.** 50 permutações do stream e contagem de alarmes por permutação. Reporta  $p$ -valor empírico  $P(\text{null} \geq \text{observado})$ . Sem esta etapa, qualquer alarme observado seria interpretado como evidência, mesmo se compatível com ruído temporal.
- **Seção 10.2 — Latência de detecção.** Stream *bootstrap* de 2019 com shift sintético injetado em  $t^* = 200$ , varrendo  $\Delta \in \{0, 1, 2, 5, 10, 20, 50\}$  mm. Para cada par (detector,  $\Delta$ ) reporta `lat_p10`, `lat_p50`, `lat_p90`. Métrica essencial para o Mês 3, que decidirá quando acionar retreino.
- **Seção 11 — Decomposição covariate vs concept.** Resposta para a pergunta central da IC:

$$\underbrace{\text{ADE}_y - \text{ADE}_{2019}}_{\text{excess total}} = \underbrace{\text{ADE}_y^{\text{compat}} - \text{ADE}_{2019}}_{\text{concept}} + \underbrace{(\text{ADE}_y - \text{ADE}_y^{\text{compat}})}_{\text{covariate}}$$

onde  $\text{ADE}_y^{\text{compat}}$  é o ADE médio em jogos do ano  $y$  com `ks_speed`  $\leq K^*$  e  $K^*$  é a mediana de `ks_speed` em 2019. Saída em `drift_decomposition.csv`.

- **Comparativo Seq2Seq vs Kalman.** ADWIN/Page-Hinkley rodam em ambos os streams. Diferença entre os dois isola miscalibração do modelo do drift do alvo.

## 5 Estado atual do pipeline

A Tabela 2 resume a cadeia de produção. O fluxo completo está documentado em `Relas/PIPELINE.md` com *data dictionary* de todos os artefatos.

| Origem                                                | Script                       | Destino                                                                                                                           |
|-------------------------------------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| dataset/proc_set_*.pkl                                | plot_covariate_shift.py      | covariate_shift_out/per_game_context.csv<br>covariate_shift_out/per_window_drift.csv<br>covariate_shift_out/ks_wd_per_dataset.csv |
| robot_30_15_t.weights.h5<br>normalization_stats_*.pkl | compute_trajectory_errors.py | covariate_shift_out/trajectory_errors_samp                                                                                        |
| artefatos acima                                       | drift_analise.ipynb          | Relas/results/mes1/*<br>Relas/results/mes2/*<br>dataset_enriched.csv                                                              |

Table 2: Cadeia de produção do pipeline.

## 5.1 Como reproduzir

```

pip install -r requirements.txt
python plot_covariate_shift.py
python compute_trajectory_errors.py --n_per_year 3 --seed 42
jupyter nbconvert --to notebook --execute drift_analise.ipynb \
    --output drift_analise.executed.ipynb \
    --ExecutePreprocessor.timeout=600

```

## 5.2 Dependências novas

`river` == 0.21.2 (detectores online) e `ruptures` == 1.1.9 (detecção offline) foram adicionadas ao `requirements.txt`.

# 6 Análise da pesquisa atual

## 6.1 O que o pipeline já permite afirmar

1. Existe degradação de ADE entre 2019 (treino) e os anos subsequentes, visível tanto em granularidade anual (notebook original) quanto por jogo (Seção 4 do notebook atual).
2. A degradação tem cauda mais pesada do que média: a razão `p99/p50` do ADE cresce mais rápido do que a média (Figura 3c\_p99\_p50\_ratio). O modelo não erra “um pouco mais em tudo”; ele falha cataclismicamente em subconjuntos específicos.
3. As features cinemáticas `ks_speed` e `ks_accel` variam intra-ano (Figuras 2b\_window\_box\_\*), o que justifica granularidade fina e não só média anual.
4. ADE Seq2Seq por jogo correlaciona positivamente com `speed_p90` e `accel_p90` do jogo (Figuras 4b\_\*), sugerindo que jogos com cauda cinemática mais agressiva são sistematicamente piores para o modelo.

## 6.2 O que ainda é hipótese

1. *Quem domina: covariate ou concept drift?* A Seção 11 do notebook produz uma estimativa, mas o valor definitivo depende de rodar `compute_trajectory_errors.py` com os pesos reais (e não com o *stub* sintético usado durante o desenvolvimento) e do tamanho da amostra (3 jogos por ano é frugal demais). **Próxima ação:** aumentar para  $\geq 5$  jogos por ano e refazer a Tabela 11.
2. *Os alarmes coincidem com mudanças conhecidas da liga?* A discussão da Seção 12 lista hipóteses (hiato pandêmico 2019→2022, evolução de hardware, mudanças de regras), mas confirmação requer cruzar com o *rule-book changelog* oficial da SSL e os *Champion Team Description Papers* dos anos relevantes [7].

3. *Os detectores têm sinal acima do null?* A Seção 10.1 responde quantitativamente, mas ainda não foi rodada com o stream real — depende novamente de `trajectory_errors_sample.parquet` produzido pelo modelo verdadeiro.

### 6.3 Limitações honestas

- **Tamanho amostral.** 3 jogos por ano  $\times$  7 anos = 21 jogos efetivos (excluindo 2020). Para detectores que se beneficiam de *streams* grandes (ADWIN, Page-Hinkley), isso é o único recurso para concept drift; para covariate shift, `df_win` dá 253 janelas, o que é mais confortável.
- **Saltos artificiais entre anos.** O stream concatena anos em sequência (2019  $\rightarrow$  2021  $\rightarrow$  2022  $\rightarrow$  ...). Os detectores online vão reportar drift na *fronteira* de ano, mas isso é parcialmente artefato da concatenação não-contínua. As fronteiras estão destacadas nos plots como linhas pontilhadas para leitura honesta.
- **Calibração geralística.** Todos os detectores usam defaults da literatura, recalibrados quando visivelmente impertinentes (Page-Hinkley com `threshold = 50` era cego). Calibração específica do domínio SSL (*e.g.* via validação cruzada em *slices* sintéticos) é tarefa do Mês 3.
- **Ground truth do drift.** Não temos rótulos de “drift sim/não” para cada janela. Toda a validação é indireta: null permutacional (Seção 10.1) e injection de shift sintético (Seção 10.2).

## 7 Próximos passos — Mês 3

1. **Importance weighting.** Estimar  $w(x) = p_{\text{alvo}}(x)/p_{2019}(x)$  via KLIEP ou RuLSIF [5, 6] sobre as features cinemáticas. Recalcular ADE ponderado em 2019 e comparar com ADE não-ponderado em  $y \neq 2019$ . Se ficarem próximos, o drift é mostly covariate e o problema é solucionável sem retreino.
2. **Retreino seletivo.** Usar os *breakpoints* do Pelt como pontos candidatos a re-treino. Para cada  $t^*$ : treino = jogos antes; teste = jogos depois. *Fine-tuning* leve (camada final do Seq2Seq descongelada) e comparação de ADE pré/pós retreino.
3. **Decomposição por feature.** Hoje a decomposição covariate vs concept usa apenas `ks_speed`. Estender para `ks_accel` e combinação multivariada.
4. **Validação por divisão.** `division_map.csv` já foi criado; é hora de cortar todas as análises por divisão A vs B e ver quanto da variação por ano é confundida com variação por divisão.

## Lista de artefatos atuais

- **Scripts:** `plot_covariate_shift.py`, `compute_trajectory_errors.py`.
- **Notebook:** `drift_analise.ipynb` (55 células).
- **Documentação:** `Relas/PIPELINE.md`, `Relas/CODEX_CHECKLIST.md`, este arquivo (`Relas/checkpoint.tex`).
- **Configuração auxiliar:** `division_map.csv`, `requirements.txt` (atualizado com `river`, `ruptures`).
- **Saídas Mês 1:** 19 pares de PDF (PT/EN) e 3 CSVs em `Relas/results/mes1/`.
- **Saídas Mês 2:** 12 pares de PDF e 6 CSVs em `Relas/results/mes2/`.

## References

- [1] A. Bifet and R. Gavaldà. Learning from time-changing data with adaptive windowing. In *SIAM International Conference on Data Mining (SDM)*, 2007.
- [2] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

- [3] C. Raab, M. Heusinger, F.-M. Schleif. Reactive Soft Prototype Computing for Concept Drift Streams. *Neurocomputing*, 416:340–351, 2020.
- [4] C. Truong, L. Oudre, N. Vayatis. Selective review of offline change point detection methods. *Signal Processing*, 167:107299, 2020.
- [5] M. Sugiyama, S. Nakajima, H. Kashima, P. von Büna, M. Kawanabe. Direct importance estimation with model selection and its application to covariate shift adaptation. In *NIPS*, 2008.
- [6] M. Sugiyama and M. Kawanabe. *Machine Learning in Non-Stationary Environments: Introduction to Covariate Shift Adaptation*. MIT Press, 2012.
- [7] RoboCup Small Size League. Rules of the RoboCup Small Size League.  
<https://ssl.robocup.org/league-software/>